

Communication and Runtime Engineering for Two-Party ResNet Inference in HPMPC

ABY2 with HE and silent-OT preprocessing

Technical report, HPMPC `dbg` branch

hpmc 4dd999b, ConvTriple 3e80748, flexNN ce50579

September 29, 2026

Abstract

We describe the optimizations and fixes that make HPMPC’s two-party ABY2 inference of ResNet50 fast, reproducible and correct across all of its protocol variants. The work targets both communication and runtime, in the preprocessing phase (homomorphic encryption for linear layers, silent OT for Boolean material) and in the online phase (round latency and per-round work). For one ImageNet image, preprocessing drops from 17.1 s to 4.3 s on two AMD EPYC 7543 hosts (3.2 s on EPYC 9354) and its triple traffic from 1112 to 587 MiB. The online phase drops from 2.4–4.3 s to 0.70 s. BatchNorm triples shrink from 5.7 GB to 0.19 GB of traffic per 10 CIFAR-10 images, and Beaver 3- and 4-tuples need half the OTs and about 45% less traffic. On CIFAR-10 we evaluate 30 protocol variants (three MSB adders, five ReLU families, fused and unfused BatchNorm) in single-batch and 192-image multi-batch mode on two machine pairs. Per 10 images, preprocessing takes 1.6–3.1 s and the online phase 0.35–0.52 s; 192 images take 12.5–24.4 s of preprocessing. With public weights, which need no linear-layer triples, preprocessing takes 0.9–2.0 s per 10 images. With the AdamW-trained ResNet50, every variant classifies 62–73 of 100 images (plaintext 76) and 60–69% of 192 images (plaintext 71.9%). Along the way we fixed a thread race, two connection hangs, two protocol bugs that broke whole variant families, and the sources of run-to-run nondeterminism: all 60 threaded builds now reproduce their serial reference bit for bit.

1 Introduction

Two-party inference with a preprocessing phase splits the cost of a secure neural network into two parts. An input-independent *preprocessing phase* produces correlated randomness: multiplication triples for the linear layers, generated with homomorphic encryption (HE), and Boolean triples and tuples for the nonlinear layers, generated with oblivious transfer (OT). The *online phase* then evaluates the network on masked values and needs only a few messages per layer. HPMPC implements this design with an ABY2-style protocol [1] (PROTOCOL=4) and generates its preprocessing material with the CHEETAH [2] HE and silent-OT [3] machinery (ConvTriple).

This report collects the engineering work that took this pipeline from a working prototype to a fast and reproducible system. Its focus is communication and runtime; Fig. 2 summarizes the largest effects.

- **HE preprocessing** (Section 3): a packed and pipelined convolution evaluator, an exact wire format that sends only the bits decryption needs, and BatchNorm triples packed into full ciphertexts and generated for all layers in one product. ImageNet preprocessing: 17.1 s $\rightarrow$ 4.3 s (4.0 $\times$).
- **Boolean preprocessing** (Section 4): OT packs sized by total demand, ferret parameters and a prefetching LPN step, TCP_NODELAY on every channel, bit-packed random OTs, and Beaver tuples whose products share one OT per common factor (half the OTs, $\approx$ 45% less traffic).
- **Online phase** (Section 5): Nagle-free sockets, element-parallel circuit levels for batch size one, pooled GEMM threads, an ahead-of-time PRNG and fewer rounds per ReLU. ImageNet online: 2.4–4.3 s $\rightarrow$ 0.70 s.
- **Correctness and reproducibility** (Section 7): deterministic OT and HE randomness, and fixes for a worker race, lost wake-ups, stale listeners, multi-batch A2B and public-weight BatchNorm.
- **Evaluation** (Section 8): 30 variants $\times$ 2 batch modes $\times$ 2 machine pairs, with final accuracy on a trained model.

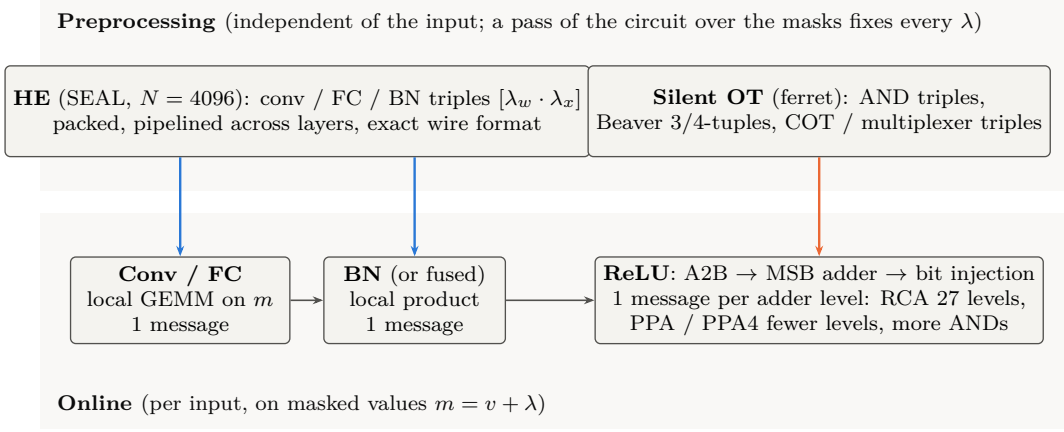


Figure 1: Where the cost of 2PC inference arises. A preprocessing pass fixes every mask; HE produces the linear-layer triples and silent OT the Boolean material. Online, a linear layer costs one message and a ReLU one message per level of its MSB adder.

	flare / polynize	algofi / goracle
CPU	AMD EPYC 9354 (Zen 4), AVX-512, VAES, IFMA	AMD EPYC 7543 (Zen 3), AVX2, AES-NI
Cores, memory	32 cores / 64 threads, 752 GB	32 cores / 64 threads, 512 GB
Link	25 Gbit/s direct (Intel E810, ice 2.6.7), MTU 9700, 24.7 Gbit/s iperf3, RTT 0.28–0.31 ms	
Roles	P0 (model owner): flare / algofi; P1 (data owner): polynize / goracle	

Table 1: Machines. Each host builds its own binaries (`-march=native`).

2 Setting

Protocol. ABY2 shares a value v as a public masked value $m = v + \lambda$ together with additive shares of the mask $\lambda = \lambda_0 + \lambda_1$. Every mask is fixed in preprocessing, so a linear layer is local except for one message that re-masks its output, and its preprocessing is a triple over the masks, e.g. $[\lambda_w \cdot \lambda_x]$ for a convolution (Fig. 1). With `A_KNOWN=1` the model owner P0 holds the weight masks and the data owner P1 the activation masks (the AB2 triple); with `A_KNOWN=0` both are shared (AB). A ReLU converts its input to Boolean shares (A2B), extracts the sign with an MSB adder and injects the bit back into the arithmetic domain (bit injection). The adder sets the number of online rounds per ReLU: the ripple-carry adder (RCA) needs 27 AND levels, the parallel-prefix adders PPA and PPA4 (4-input prefix gates, which use Beaver 3- and 4-tuples) need fewer levels but more AND gates.

Variants. We measure three adders (RCA, PPA, PPA4) and five ReLU families: *plain*; *reshare* (`RESHARE_OPT`); *reshare+sim*, which folds the reshare material into the convolution mask (`RESHARE_OPT_SIM`); *a2b*, which moves the A2B to preprocessing and bakes its mask into the convolution output (`A2B_ONLINE_OPT`, `A2B_CONV_BAKE`); and *a2b+AKTE*, an adder for inputs whose mask the evaluators know (`A_KNOWN_TO_EVALUATORS_OPT`). Each variant runs with BatchNorm fused into the preceding convolution or unfused. All variants share `BITLENGTH=32`, `FRACTIONAL=5`, probabilistic (SecureML-style) truncation, `FUSE_RELU_AVG=1` and `CUT_FRACTIONAL_BITS_OPT=1`.

Workloads. ResNet50 on CIFAR-10 in two modes. *Single batch*: one process, 32-bit lanes (`DATYPE=32`), 10 images, 32 HE/OT threads and 31 extra online threads. *Multi-batch*: 24 processes with 256-bit registers of eight 32-bit lanes, 192 images, 3 HE/OT threads and one extra online thread per process. The HE preprocessing study uses ResNet50 on one 224×224 ImageNet image.

Measurement. Preprocessing and online time are P0’s wall-clock times (the maximum over processes in multi-batch). Single-batch numbers are medians of two runs; multi-batch runs once. All randomness is seeded, so every run of a build produces a bit-identical output; we check each run’s output hash against a serial build without extra threads (Section 7). The runtime does not depend on the weights: HE triples are generated with `TRIPLE_ZERO=ON` and no code path skips zero values. Accuracy uses two trained ResNet50 models: `Cifar_adam_001` for the runtime matrix and an AdamW-trained model (74.35% on the test set) for the final accuracy.

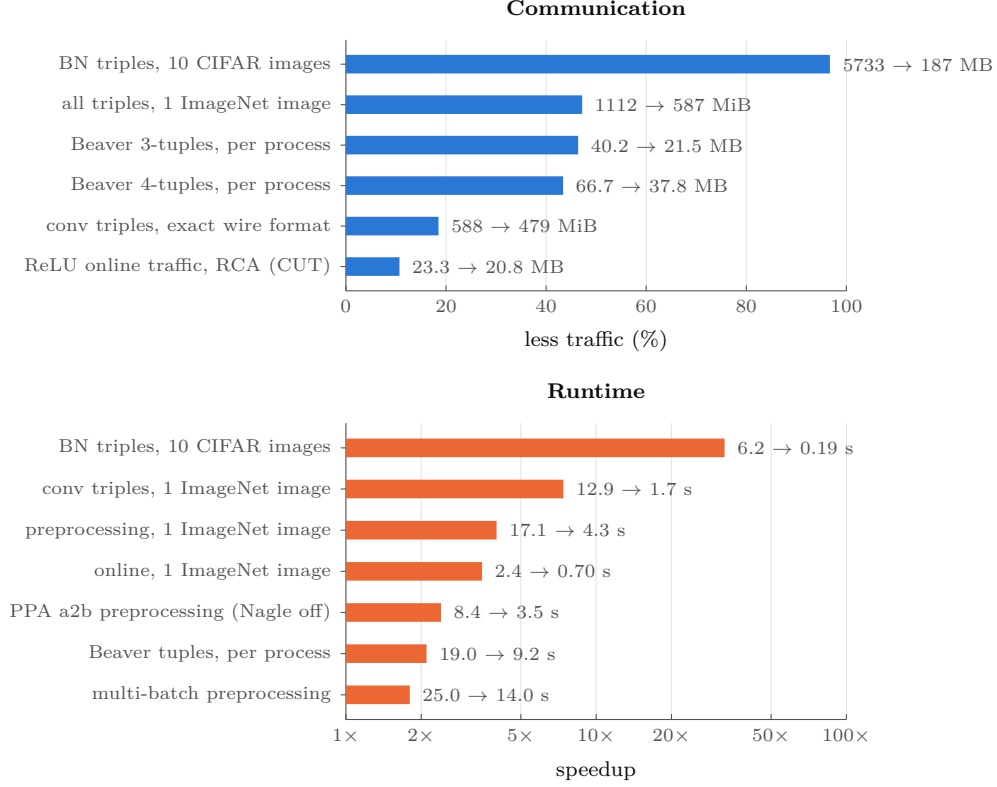


Figure 2: Headline effects of the changes in this report, each labeled with its before and after values. Communication: share of traffic removed. Runtime: speedup (log scale). Sections 3–6 explain each.

3 Linear layers: HE preprocessing

Figure 3 traces ImageNet preprocessing through the ten ConvTriple changes (s1–s10); Table 2 lists what each step does.

Packed evaluator (s1, s2). CHEETAH’s convolution encrypts one input tile per ciphertext and returns one ciphertext per output channel and tile. The packed layout of the GPU path puts several images and output channels into each ciphertext: triple traffic falls from 1112 to 698 MiB. At first this made the evaluator slower (s1), because every weight polynomial still took a full NTT. The evaluator now transforms a weight polynomial right before its only use, and it transforms 1×1 weights with NTTs of size $N/2^a$. It builds sparse polynomials from a table of powers and sums products lazily in 128 bits. Convolution time falls from 13.7 to 7.5 s.

Randomness and threads (s3–s6). Flooding noise and masks come from an AES-CTR generator (AES-NI) instead of SEAL’s Blake2; the thread pool survives across the several parallel calls of a layer; the multiplexer and COT triples run on all threads. Together these steps take preprocessing from 11.9 to 5.4 s.

Pipelining (s8). Per layer, each party idled while the other worked: at s7 P0 waited 0.82 s and P1 1.95 s in total. The pipelined protocol (Fig. 4) overlaps P1’s encryption and decryption with P0’s evaluation. It sends exactly the same bytes and cuts convolution time from 2.6 to 1.9 s; with $A_KNOWN=0$, where both parties encrypt, from 3.9–4.0 to 2.0 s in isolation.

Exact wire format (s9). SEAL serializes a seeded input ciphertext with zstd to 62.4 KB. P1 now sends only a 16-byte seed and c_0 at exactly $60 + 49 = 109$ bits per coefficient (55.8 KB). P0 expands c_1 from the seed directly in NTT form, which saves an inverse and a forward NTT per input. For outputs, the modulus switch before decryption keeps 34 bits of c_0 and 46 of c_1 ; exactly those bits travel, 20–25% less than zstd. The tile search now minimizes bytes rather than ciphertexts, since an input costs about 2.4 times an output. Convolution traffic falls 18% and all preprocessing traffic 12% (Table 3); decrypted values are bit-identical, and a check rejects outputs whose dropped bits are not zero. The 109-bit input modulus is a floor: BFV’s plaintext product carries a term of up to $\approx 2^{74}$.

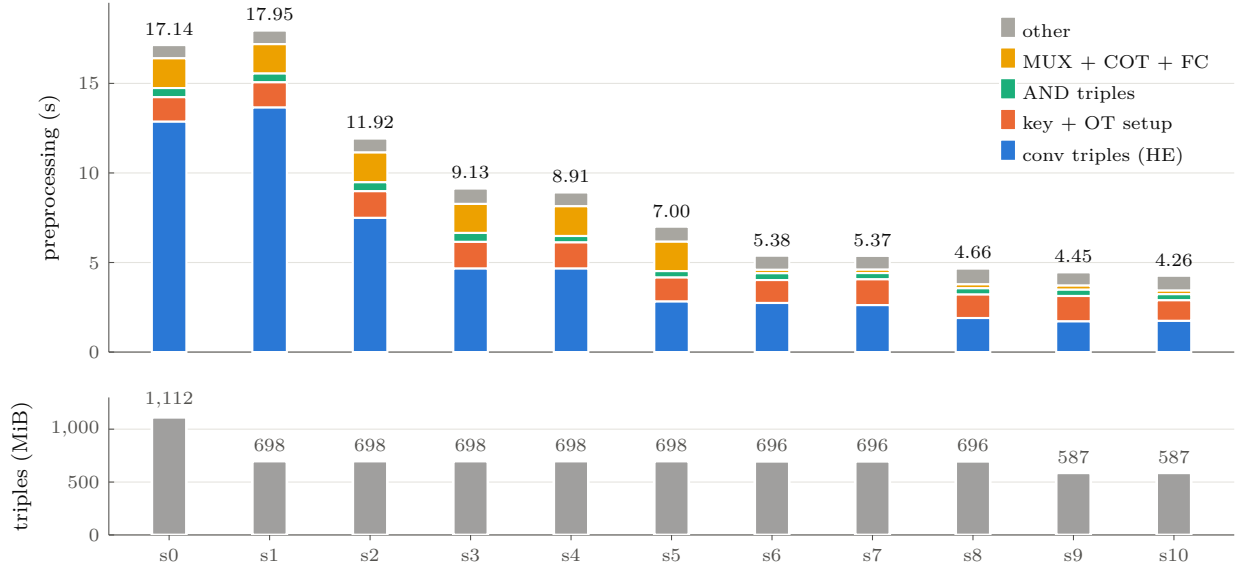


Figure 3: Preprocessing of one ImageNet image on algofi / goracle, by component (median of 3 runs), and the triple traffic sent plus received by P0. s2–s6 are value-independent reruns (Section 3).

Step	Change	Pre (s)	Conv (s)
s0	Baseline: CHEETAH’s HomConv2DSS, one output ciphertext per channel and tile	17.14	12.86
s1	Packed layout: several images and output channels per ciphertext (traffic 1112 → 698 MiB)	17.95	13.65
s2	Packed evaluator: weight polynomials transformed right before use, 1×1 weights through NTTs of size $N/2^a$, sparse polynomials from a power table, lazy 128-bit accumulation	11.92	7.49
s3	AES-CTR noise and masks instead of Blake2; ciphertexts loaded in parallel	9.13	4.66
s4	AND triples from bit-packed random OTs (64 per word)	8.91	4.66
s5	Thread pool kept across calls; multiply-accumulate blocked per prime	7.00	2.82
s6	Multiplexer and COT triples on all threads (MUX 1.08 → 0.07 s, COT 0.48 → 0.04 s)	5.38	2.74
s7	All work independent of the weights’ values (TRIPLE_ZERO=ON)	5.37	2.62
s8	Convolutions pipelined across layers (Fig. 4)	4.66	1.90
s9	Exact wire format and byte-optimal tiling (conv traffic 588 → 479 MiB)	4.45	1.71
s10	Prefetching LPN step, ferret parameters b12 (key setup 1.42 → 1.13 s)	4.26	1.74

Table 2: The HE and OT preprocessing steps (ImageNet, algofi / goracle, 32 threads). The same build preprocesses in 3.2–3.3 s on flare / polynize.

BatchNorm triples. Unfused BatchNorm multiplies each activation by a per-channel scale, which the slot-encoded protocol computes as an elementwise product of two vectors. The original code split each image’s elements among the 32 threads before packing: every thread sent its own nearly empty ciphertexts, one image at a time. Packing all images into full ciphertexts cut BN traffic from 2704 / 3029 to 90 / 101 MB and BN time from 6.2–7.9 to 0.7–0.8 s (CIFAR-10, 10 images). Each of the 53 BN layers was still a separate call, and one call is latency-bound (1.6–6.4 MB in 13–50 ms). CHEETAH_BN_BATCHED concatenates all slot-encoded layers into one product, split among the threads in whole ciphertexts: the BN phase falls from 0.75 to 0.19 s with the same traffic (Table 4).

Value independence. The benchmark’s dummy model makes every weight mask zero, and early versions of the packed evaluator skipped zero weights, so their benchmark times looked 0.6–2.9 s faster. With weights known in preprocessing, a zero-skipping evaluator would also leak the number of zero weights through its timing. All evaluators now do the same work for any weight values, and all numbers in this report are value-independent.

4 Boolean material: OT preprocessing

The ReLUs consume AND triples, Beaver 3- and 4-tuples (PPA4), COT triples (bit injection) and multiplexer triples. All come from ferret silent OT, 32 OT packs of two ferret instances each.

Ferret parameters and the LPN step. With 32 instances at once, ferret’s LPN step is memory-bound on its 7.2 MB tables. Computing the AES of the next four groups ahead and prefetching their table rows cuts the

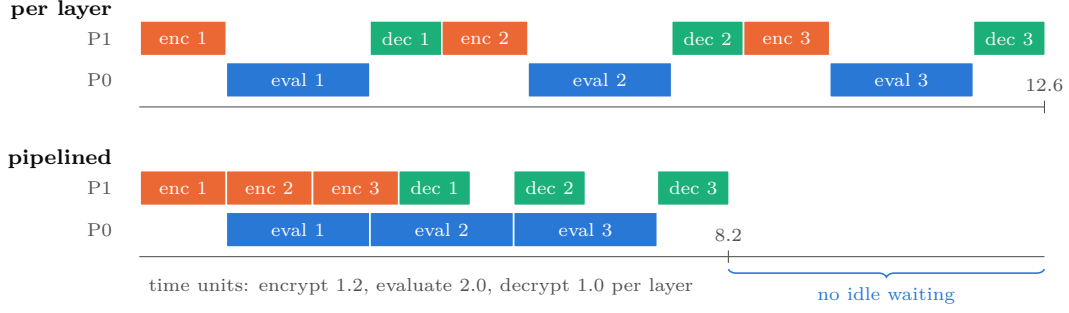


Figure 4: Pipelining the convolutions across layers (s8). Per layer, P1 waits while P0 evaluates and vice versa; pipelined, P1 encrypts the next layer’s inputs and decrypts the previous layer’s outputs while P0 evaluates. Same bytes, preprocessing 5.69 → 4.66 s.

MiB, P0 sent / received	before (s8)	after (s10)
Conv triples	232.6 / 355.1	234.9 / 244.1
Ferret setup	46.2 / 46.2	52.7 / 52.7
Multiplexer triples	35.4 / 35.4	35.4 / 35.4
COT triples	34.4 / 1.1	34.4 / 1.1
FC triples	0.7 / 1.2	0.7 / 1.2
Total	349.4 / 439.0 (788.4)	358.2 / 334.6 (692.8)

Table 3: Traffic of one ImageNet preprocessing (A_KNOWN=1). With A_KNOWN=0 the triple traffic falls from 659 / 626 to 551 / 518 MiB.

step from 0.34 to 0.28 s with bit-identical outputs. Parameter set b12 has a 3.8 MB table that stays in cache, for 6.5 MB more setup traffic per direction. Key setup falls from 1.42 to 1.13 s. VAES did not help: the LPN step is memory-bound and MITCCRH is bound by its AES key schedule.

Demand-sized OT packs. The first OT consumer used to size the packs, and later consumers extended them in small steps. hpmc now passes the total OT demand of the whole preprocessing to ConvTriple (ot_demand_hint) before the first generation. On CIFAR-10 the OT-pack phase falls from 0.91 to 0.47 s, and PPA4, reshare and a2b get enough capacity for their tuples.

TCP_NODELAY everywhere. The ConvTriple channels enabled TCP_NODELAY only in WAN mode. The COT multiply behind the a2b bake flushes small messages 31 times in sequence, and each flush waited for a delayed ACK. One call took 170 ms; with Nagle off it takes 11 ms. PPA a2b single-batch preprocessing falls from 8.4 to 3.5 s, PPA4 reshare+sim from 8.9 to 5.0 s. The same fix on hpmc’s own sockets takes the ImageNet online phase from 2.4–4.3 to 1.9–2.2 s (Section 5).

Bit-packed random OTs. Random OTs with one-bit messages are hashed and reduced to bits in cache-sized chunks and stored eight per byte (send/recv_rot_bits), where the generic interface kept two 16-byte blocks per OT. AND triples take 0.36 instead of 0.50 s, and the COT multiply reads the same bits with far less memory traffic.

Beaver 3- and 4-tuples with shared OTs. A 3-tuple $(a, b, c, ab, ac, bc, abc)$ was generated as an AND triple (a, b, ab) plus three independent share multiplications $a \cdot c$, $b \cdot c$ and $ab \cdot c$. Each multiplication took one random OT per direction: the receiver sent a choice correction $e = s \oplus x$, the sender a masked value $y \oplus r_0 \oplus r_1$. The three products share the factor c . A random OT’s messages are hashes of 128-bit blocks, so one OT can carry k message bits for the price of one (send/recv_rot_bitplanes). cot_outer_multiply computes all $x_p \cdot y$ for k values x_p from one OT per direction, sending k masked bits and at most one correction bit. Because c is fresh, each party takes its share of c from the random choice bits of its receiving OTs and sends no correction at all (Fig. 5). A 4-tuple needs the nine products $\{a, b, ab\} \times \{c, d, cd\}$: three OTs per direction instead of nine. Per tuple, AND triples included, a 3-tuple now needs 4 random OTs instead of 8 and a 4-tuple 10 instead of 22. In a multi-batch PPA4 run the tuple phases fall from 19.0 to 9.2 s per process and their traffic by 43–46% (Fig. 6). The party-local variant used by reshare+sim, where P0 holds b and P1 holds c in full, is covered by the same code with one correction bit for P0’s zero share. Unit tests check all products; ReLU exactness tests report 0 wrong outputs.

CIFAR-10, 10 images	traffic P0 sent / recv (MB)	BN phase (s)	preprocessing (s)
threads split each image	2704 / 3029	6.2–7.9	8.2–10.0
full ciphertexts	90 / 101	0.7–0.8	3.0–3.2
all 53 layers in one product	88 / 99	0.19	1.94 (RCA)

Table 4: BatchNorm triples with unfused BN. In multi-batch the per-process BN phase falls from 1.16 to 0.85 s (3 threads per process, compute-bound).

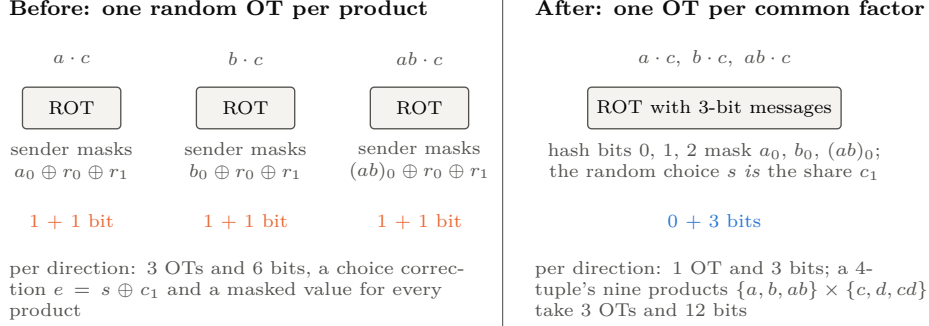


Figure 5: Products with a common factor share one random OT. Before, each product $x \cdot c$ took one random OT per direction plus two sent bits. Now one OT per direction carries all k products in its k message bits, and a party whose share of c is fresh takes it from the OT’s random choice bit.

5 Online phase

On a 0.3 ms LAN, the online phase is bound by round latency and per-round work, not by bandwidth. Activations send about 180 MB per ImageNet inference, 0.06 s at 25 Gbit/s. Figure 7 shows the steps for one ImageNet image.

Nagle, buffers and GEMM threads. With Nagle’s algorithm, a round’s last small message waited for a delayed ACK. ReLU layers were bimodal (6 or 46 ms), and the online phase varied between 2.4 and 4.3 s. With TCP_NODELAY it takes 1.9–2.2 s. Send and receive buffers of 100 000 words mean fewer messages and thread hand-offs per round. The threaded GEMM first sent its outputs in linear order while the completion visits 64×64 tiles; CIFAR-10 accuracy was 0% until both used the tile order. Its threads now live in a lock-free pool.

Element-parallel levels at batch size one. HPMPC vectorizes only across independent inputs, so a single image evaluated every level of a ReLU (adder steps, A2B, bit injection) element by element on one thread. Within a level, every element runs the same code and consumes the same number of words from each stream: sends, receives, preprocessed values, its own random words and the indices into the triple buffers. `stream_parallel_for` measures this consumption on the first element and runs the rest on the worker pool, each worker with cursors at exactly the stream positions the serial loop would use (Fig. 8). Segments end at send/receive buffer boundaries, and a misuse check verifies every worker’s consumption. The messages, their order and all values are those of the serial loop. With bulk random words, parallel adder construction and the GEMM mask-and-send and completion on the same pool, the ImageNet ReLU time falls from 630 to 300 ms and the online phase from 1.09 to 0.70 s. `RNG_AHEAD` moves the party’s own sequential AES stream to a background thread.

Rounds per ReLU. What remains is round latency: 1,752 rounds for CIFAR-10 with RCA against 776 with PPA and 625 with PPA4 (Table 5). `CUT_FRACTIONAL_BITS_OPT` removes the fractional bits from the MSB adder by substituting identity elements and public constants. RCA loses 245 of its 1,997 rounds (12%), and ReLU traffic falls 8–11% for every adder. Preprocessing and accuracy do not change measurably (1,134 vs. 1,138 of 1,752 evaluations correct with and without CUT).

6 Multi-batch

With 24 processes of 256-bit lanes, preprocessing dominates. With more than one HE/OT thread per process, the processes shared connections: channel i used port $base + i \cdot PROCESS_NUM$ while process bases stepped by 3, which caused hangs and wrong triples. With a stride of $3 \cdot PROCESS_NUM$, preprocessing falls from

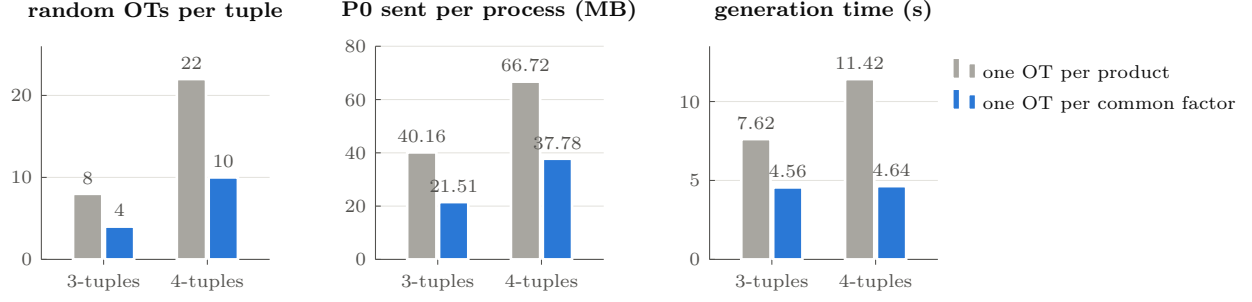


Figure 6: Beaver tuple generation per process in a multi-batch PPA4 run (flare / polynize, 24 processes).

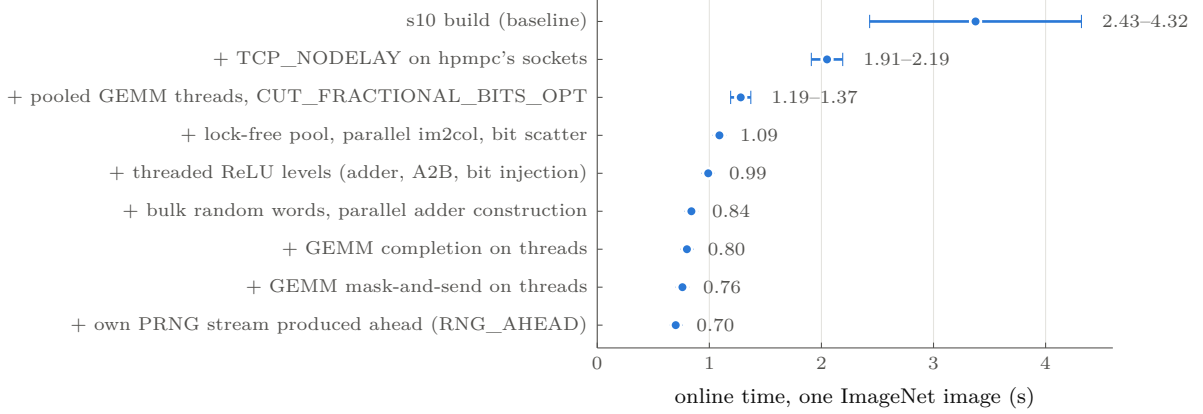


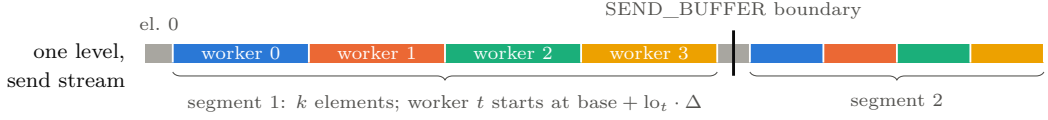
Figure 7: Online phase of one ImageNet image on flare / polynize (ranges over runs where they varied). Output hashes are unchanged by every step.

25.0 to 14.0s with three threads per process (Table 6). One extra online thread per process cuts the online phase from 0.81 to 0.67s with an identical output hash.

7 Correctness and reproducibility

Fast code is only useful if its outputs are right and repeatable. We made every run bit-reproducible and compared each threaded build against a serial one; this exposed the following bugs.

- **Nondeterministic OT.** COT and multiplexer triple shares are the ferret outputs, and hpmc truncates shares locally, so their per-run randomness changed the outputs. ConvTriple’s PRGs are now seeded per party, stage, call and task, and a scoped override seeds emp’s default PRGs per OT pack and worker (DetSeedScope).
- **Worker race in P0’s A2B.** For the reshare+sim skip with PPA4, P0 chooses each group’s slice masks from the Beaver 3-tuples that the group’s adder will consume, found through a count of groups prepared so far. On the worker pool that count was one shared global. Threaded PPA4 reshare+sim with fused BN gave a different output every run. The count is now a per-worker stream index. The race stayed hidden whenever the pool threads woke one after another.
- **Hangs.** A lost wake-up at connection start, where flag and broadcast were not under the waiters’ mutex, and a stale listener into whose backlog the next phase’s peer connected.
- **Multi-batch a2b.** The A2B mask bake derived the conv mask by casting transposed words to lanes, which holds only for 32-bit lanes; with 256-bit registers the a2b families classified 13–29 of 192 images. The mask is now computed value by value with the A2B’s own inverse packing, which leaves single-batch hashes unchanged, and the multi-batch a2b families classify 99–132 of 192 images, like the plain family.
- **Public weights with unfused BN.** With public weights the first convolution’s output is still the data owner’s sharing (the other party’s mask is zero), and SecureML truncation of that sharing wraps for positive values: 0 of 10 correct. The owner now truncates it exactly and re-masks, in the same single message.
- **Test harness.** The ReLU tests fed unbaked inputs to the reshare+sim skip (35% wrong); the harness now declares its inputs unbaked.



1. Element 0 runs serially and is measured: Δ = the words it sent, received and read from each preprocessing stream, the random words it drew, and its advances of the triple indices.
2. The other elements run on the pool, each worker with its own cursors into every stream; elements next to a buffer boundary run serially, and every worker's consumption is checked.
3. Messages, their order and their split into buffers are exactly those of the serial loop.

Figure 8: stream_parallel_for: element-parallel circuit levels at batch size one. One element is measured; the level's remaining elements run on the worker pool with per-worker cursors into every stream, split at buffer boundaries.

single batch, unfused BN	rounds		ReLU traffic (MB)		online (s)	
	CUT	no CUT	CUT	no CUT	CUT	no CUT
RCA	1,752	1,997	20.8	23.3	0.52	0.48
PPA	776	789	32.6	36.8	0.44	0.49
PPA4	625	625	20.1	21.8	0.42	0.42

Table 5: Online rounds, P0's ReLU traffic and online time for 10 CIFAR-10 images on flare / polynize, with and without CUT_FRACTIONAL_BITS_OPT. The time differences are within run-to-run noise.

After these fixes, all 60 threaded variants (30 single-batch, 30 multi-batch) reproduce their serial build's output hash on flare / polynize, and repeated runs are bit-identical. Hashes differ between the machine pairs, because the PRNG draws 512-bit AES blocks with VAES-512 on Zen 4 and 256-bit blocks on Zen 3.

8 Evaluation of all variants

Figure 9 shows preprocessing and online time for every variant on both machine pairs.

Single batch. On flare / polynize, preprocessing takes 1.6–2.2 s with RCA, 2.3–2.5 s with PPA and 2.5–2.8 s with PPA4 when BN is fused, and 1.9–2.4, 2.6–2.9 and 2.7–3.1 s unfused; the a2b families sit at the upper end. The online phase takes 0.35–0.52 s. Because it is round-latency-bound, PPA and PPA4 are slightly faster online than RCA despite more AND gates. The fastest configuration in total is RCA with fused BN (1.66 s + 0.40 s).

Multi-batch. 192 images take 12.5–16.6 s (RCA), 15.7–19.8 s (PPA) and 18.7–24.4 s (PPA4) of preprocessing and 0.54–0.79 s online. Per image that is 65–127 ms of preprocessing, against 166–310 ms in single batch.

Machines. In single batch, algofi / goracle is 1–16% slower in preprocessing than flare / polynize and about equal online. Multi-batch is compute-bound, and there Zen 3 needs 1.4–1.7× as long: it lacks AVX-512 for the HE NTTs and the GEMM.

The last two optimizations. Figure 10 isolates the effect of the shared-OT tuples and the batched BN triples. PPA4 gains most: single batch 4.35 → 2.68 s unfused and 3.46 → 2.61 s fused, multi-batch 31.5 → 22.6 s and 30.2 → 20.2 s. Unfused RCA gains 2.51 → 1.94 s from the batched BN.

8.1 Fully shared masks and public weights

Two further settings change who holds the weights. With `A_KNOWN=0` both parties hold shares of the weight masks as well as of the activation masks, so the conv, FC and BN triples are AB triples for which both parties encrypt. With public weights (`PUBLIC_WEIGHTS=1`) the linear layers multiply locally and need no triples at all. We run public weights with `TRUNC_DELAYED=1` and `BIT_INJECTION_TRUNC_SIM=1`: each linear layer's truncation is deferred and folded into the next ReLU's bit injection instead of costing a round of its own. Figure 11 compares both settings with the secret-weight default for the plain family.

Public weights. Only the ReLU material remains to be preprocessed: across all 30 variants, preprocessing takes 0.92–2.03 s per 10 images and 3.5–14.3 s per 192 images, 27–53% and 38–75% less than the same variant with secret weights. The online phase takes 0.27–0.41 s and 0.33–0.42 s (5–29% and 29–50% less): convolutions send nothing, and the truncation shares the bit injection's message. With the AdamW model, the public-weight

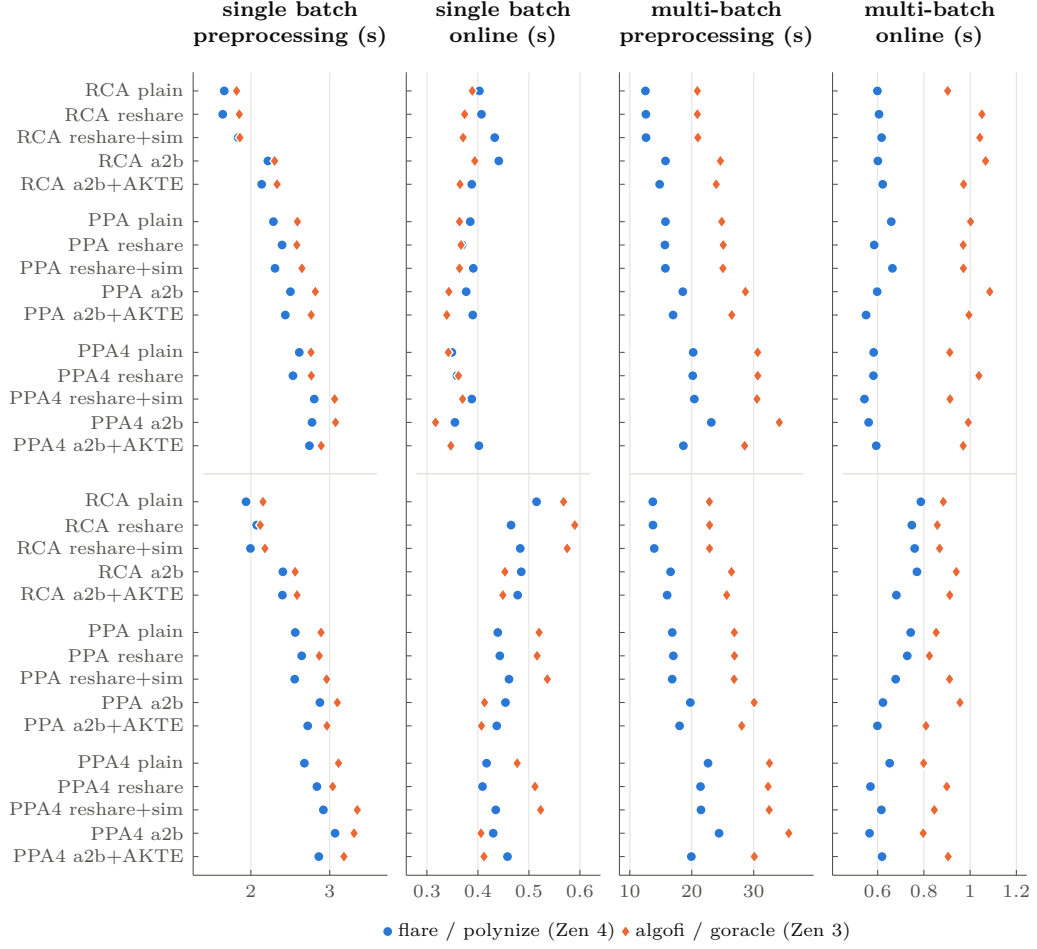


Figure 9: All 30 variants, CIFAR-10 ResNet50 (final builds). Upper block: fused BN; lower block: unfused BN. Single batch: 10 images, median of two runs. Multi-batch: 192 images, 24 processes. Tables with every value: `docs/VARIANT_PERFORMANCE.md`.

HE/OT threads	preprocessing (s)	conv (s)	AND triples (s)	BN (s)	online (s)
1	25.0	16.5	3.6	2.1	0.63
2	17.0	10.2	3.3	1.4	0.68
3	14.0	8.8	3.0	1.2	0.75
4	14.0	7.8	3.8	1.0	0.69

Table 6: Multi-batch CIFAR-10 (24 processes, 192 images, RCA, unfused BN) on flare / polynize, by HE/OT threads per process, after the port-stride fix.

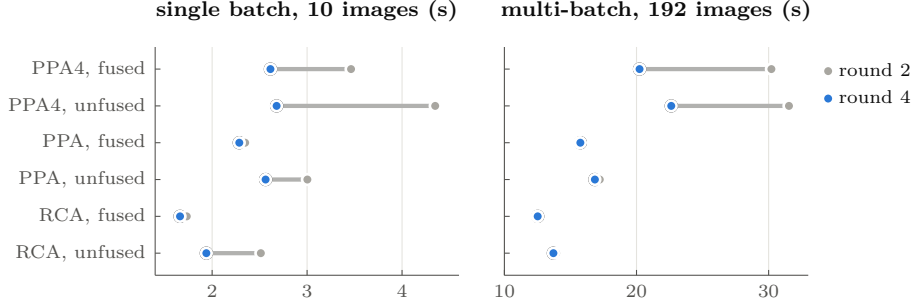


Figure 10: Preprocessing of the plain family before (round 2) and after (round 4) the shared-OT tuples and the batched BN triples (flare / polynize).

variants classify 63–73 of 100 images in single batch and 115–131 of 192 (60–68%) in multi-batch. Unfused BatchNorm needed the first-layer truncation fix of Section 7; before it, those variants classified 0 of 10 images.

Fully shared masks. With $A_KNOWN=0$ the linear-layer triples are AB triples, and preprocessing traffic grows by a factor of 1.4–1.9 (for RCA, unfused BN, 10 images: 229 to 428 MB). In single batch the preprocessing is nevertheless 3–18% faster (1.50–2.83 s over the 30 variants): both parties encrypt, and the pipelined convolutions keep both of them busy (conv phase 0.65 to 0.48 s). In multi-batch, with three threads per process, it is 3–16% slower (13.8–26.4 s). The online phase takes 0.34–0.52 s and 0.56–0.73 s. The accuracy, however, is at chance level (0–1 of 10 in every configuration we tried, including $TRUNC_DELAYED=1$ with the truncation in the bit injection). With both masks shared, the product mask $\lambda_w \lambda_x$ is itself a full-range sharing, and the probabilistic local truncation of the layer outputs wraps. $A_KNOWN=1$ avoids this because one party holds the weight mask and the conv triple delivers its product exactly. A fix needs exact or reduced-slack truncation for ABY2, which the protocol does not implement yet.

8.2 Final accuracy

Every variant stays above 60% (Fig. 12 and Table 7). The spread between variants (62–73 of 100) is truncation noise: the variants draw different amounts of randomness, so different values wrap. The adders are exact, with 0 of 32,768 random ReLU inputs wrong for each adder. Fused and unfused BN are equally accurate with this model. With the older `Cifar_adam_001` model, probabilistic truncation in the 32-bit ring cost 10–17 points (Trio 56% against 73% in plaintext; 73% with a 64-bit ring). The AdamW model is much less sensitive to it.

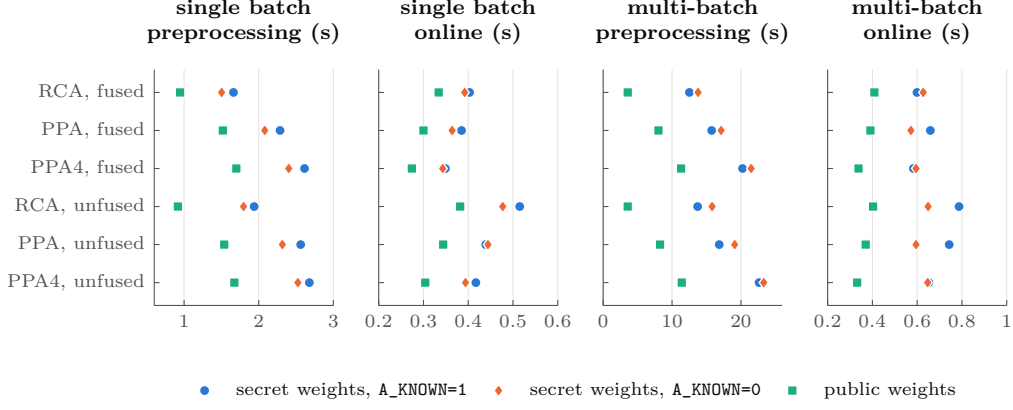


Figure 11: Secret weights with $A_KNOWN=1$ (default) and $A_KNOWN=0$, and public weights, plain family, flare / polynize. Runtime does not depend on the values, so the $A_KNOWN=0$ times are valid although its accuracy is not.

Setting	images	correct
Plaintext, float (PyTorch)	100 / 192	76 / 138 (71.9%)
Trio 3PC, standard configuration	100	71
Trio 3PC, reduced-slack truncation ($TRUNC_APPROACH=1$)	100	70
2PC ABY2, single batch, all 30 variants	100	62–73
2PC ABY2, multi-batch, all 30 variants	192	116–132
2PC ABY2, CUT on / off (plain family, 12 runs each)	1,752	1,134 / 1,138

Table 7: Final accuracy with the AdamW-trained ResNet50 on the first CIFAR-10 test images.

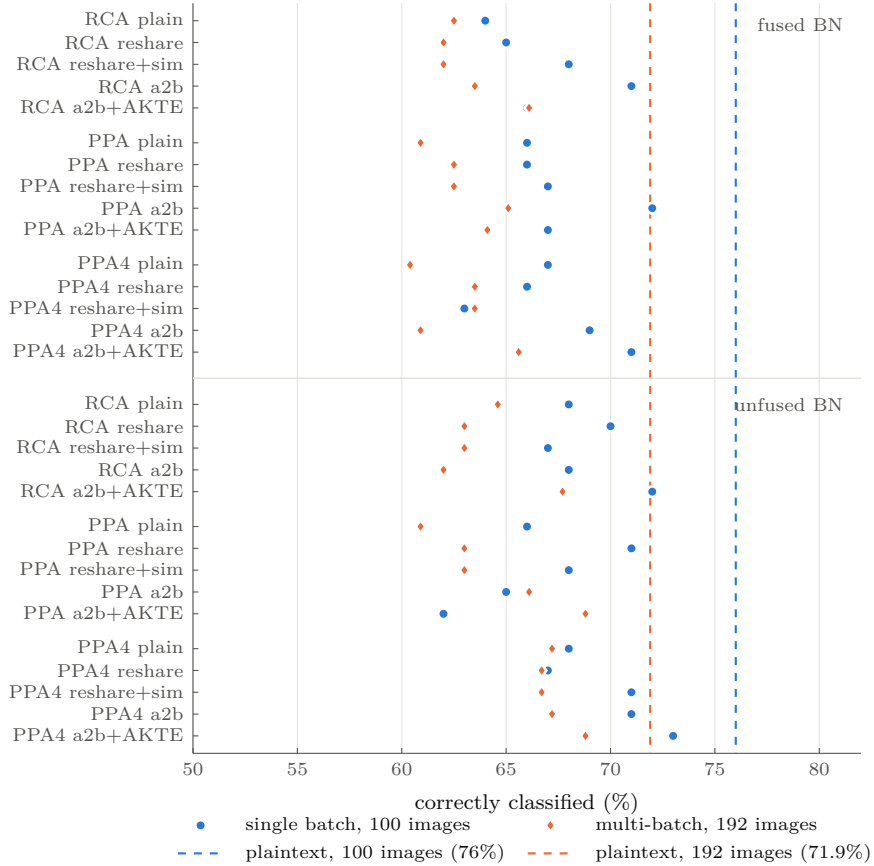


Figure 12: Accuracy with the AdamW-trained ResNet50, standard probabilistic truncation. Single batch on flare / polynize, multi-batch on algo / goracle (a2b families: builds with the multi-batch A2B bake).

9 Limitations and open problems

- **Truncation.** ABY2 has no reduced-slack or exact truncation yet; with fully shared masks (`A_KNOWN=0`) the local truncation wraps, and ResNet50 classifies at chance (Section 8.1).
- **Weights known in preprocessing.** The multi-batch a2b bake still excludes `MODELWEIGHTS_KNOWN_DURING_PREPROCESSING`, whose prescribed shares assume one word per register.
- **Ciphertext occupancy.** At 7×7 feature maps, inputs are 5–19% data and outputs 6–24%. Packing the sparse outputs needs automorphisms and a larger ring ($N = 8192$).
- **OT extension.** Ferret’s LPN step and the AES key schedules of the correlation-robust hash are now the largest CPU costs of the Boolean preprocessing; a fixed-key hash is a security decision.

10 Conclusion

The largest runtime gains came from doing less redundant work in the HE evaluator (packing, lazy transforms, pipelining). The largest communication gains came from sending only information-bearing bits: the exact wire format, full BatchNorm ciphertexts, and OTs whose message bits carry several products. In the online phase, latency rather than bandwidth dominates, so the gains came from Nagle-free sockets, fewer rounds and parallelizing the per-round work at batch size one. Seeded, bit-reproducible runs made every change measurable and exposed the bugs in Section 7.

References

- [1] A. Patra, T. Schneider, A. Suresh, H. Yalame. ABY2.0: Improved mixed-protocol secure two-party computation. *USENIX Security* 2021.
- [2] Z. Huang, W. Lu, C. Hong, J. Ding. Cheetah: Lean and fast secure two-party deep neural network inference. *USENIX Security* 2022.
- [3] K. Yang, C. Weng, X. Lan, J. Zhang, X. Wang. Ferret: Fast extension for correlated OT with small communication. *ACM CCS* 2020.